

Full Stack Development With AI

Raghavendran V



Saturday, Sep 20, 2025
11:00 AM SGT



Advanced Computing for Executives
School of Computing

Agenda

- **Quick: Install & start MySQL on mac (Homebrew)**
- **Database Systems — overview**
- **Relational Model**
- **Example ER model**
- **Create database & tables (DDL) — MySQL commands**
- **Data Integrity & Constraints — explanation + examples**
- **SQL Basics — SELECT / WHERE / ORDER / LIMIT**
- **DDL statements (create/alter/drop)**
- **DML statements (insert/update/delete/select)**
- **Aggregate functions & GROUP BY / HAVING**
- **Combining data — JOINS, UNION, subqueries**

Quick: Install & start MySQL on mac (Homebrew)

If you already have MySQL skip ahead. Otherwise:

```
# install Homebrew if you don't have it (one-time)
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

```
# install mysql
brew update
brew install mysql
```

```
# start mysql server (background)
brew services start mysql
```

```
# secure installation (set root password, remove test user)
mysql_secure_installation
```

```
# connect as root (or the user you created)
mysql -u root -p
```

From here you are at `mysql>` prompt and can run the SQL in the examples below.

Database Systems — overview (short)

- **Database** = organized collection of data.
- **DBMS** = software that manages databases (MySQL, PostgreSQL, MongoDB).
- **Components**: storage, query engine, transaction manager, security, backup.

Types (high level)

- **Relational** (row tables, SQL) — MySQL, PostgreSQL (structured schemas, ACID).
- **Document** (JSON docs) — MongoDB (flexible schema).
- **Key-Value** — Redis (fast lookups).
- **Column-family** — Cassandra (big distributed datasets).
- **In-memory** — Redis (fast cache).

Relational Model (concept)

- Data stored in **tables** (relations).
- Each table has **columns** (attributes) and **rows** (tuples).
- Keys: **Primary Key (PK)** uniquely identifies a row. **Foreign Keys (FK)** link tables.
- Relations support **joins**, **constraints**, and **normalization** to avoid redundancy.

Example ER model (University)

Entities:

- **students** (student_id PK)
- **courses** (course_id PK)
- **enrollments** (enrollment_id PK, FK → students, courses)

Simple ER (text):

```
students (student_id PK, name, email)
courses  (course_id PK, title, credits)
enrollments (enroll_id PK, student_id FK, course_id FK, grade)
```

Create database & tables (DDL) — MySQL commands

Run these at `mysql>` prompt:

```
-- create DB and use it
CREATE DATABASE university;
USE university;
```

```
-- students table
CREATE TABLE students (
  student_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  enrolled_date DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

```
-- courses table
CREATE TABLE courses (
  course_id INT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(150) NOT NULL,
  credits INT NOT NULL DEFAULT 3
);
```

```
-- enrollments table (junction table)
CREATE TABLE enrollments (
  enroll_id INT AUTO_INCREMENT PRIMARY KEY,
  student_id INT NOT NULL,
  course_id INT NOT NULL,
  grade DECIMAL(4,2),          -- e.g. 85.50
  CONSTRAINT fk_student
    FOREIGN KEY (student_id) REFERENCES students(student_id)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_course
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
    ON DELETE RESTRICT
);
```

Notes: PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT, and FOREIGN KEY are data integrity rules enforced by the RDBMS.

Data Integrity & Constraints — explanation + examples

- **NOT NULL** prevents missing values.
- **UNIQUE** enforces uniqueness (e.g. emails).
- **PRIMARY KEY** = UNIQUE + NOT NULL.
- **FOREIGN KEY** enforces referential integrity (can't reference a non-existent parent).
- **CHECK** (MySQL 8 supports CHECK; earlier versions accept but ignore): use to validate values.
- **DEFAULT** gives default value.

Try violations:

```
-- insert valid student
INSERT INTO students (name, email) VALUES ('John Doe', 'john@example.com');

-- try to insert duplicate email -> error because UNIQUE
INSERT INTO students (name, email) VALUES ('John D', 'john@example.com');

-- try to insert enrollments with non-existing student -> FK error
INSERT INTO enrollments (student_id, course_id, grade) VALUES (999, 1, 90);
```

Errors from the server show you when constraints are violated.

SQL Basics — SELECT / WHERE / ORDER / LIMIT

Example data insertion:

```
INSERT INTO courses (title, credits) VALUES  
( 'Databases', 4), ( 'Algorithms', 3), ( 'Web Development', 3);
```

```
INSERT INTO students (name, email) VALUES  
( 'John Doe', 'john@example.com'),  
( 'Jane Smith', 'jane@example.com'),  
( 'Michael Brown', 'michael@example.com');
```

```
-- enrollments: student 1 in courses 1 and 3  
INSERT INTO enrollments (student_id, course_id, grade) VALUES  
(1,1,88.5),(1,3,92),(2,1,79),(3,2,85);
```

Simple selects:

```
SELECT * FROM students;  
SELECT name, email FROM students WHERE name LIKE 'J%';  
SELECT * FROM courses ORDER BY credits DESC;  
SELECT * FROM students LIMIT 5;
```

DDL (Data Definition Language) statements (create/alter/drop)

- `CREATE TABLE` — creates schema.
- `ALTER TABLE` — add/modify columns or constraints.
- `DROP TABLE` — remove table.

Examples:

```
-- add column
ALTER TABLE students ADD COLUMN phone VARCHAR(20);

-- change column
ALTER TABLE students MODIFY COLUMN phone VARCHAR(30);

-- drop column
ALTER TABLE students DROP COLUMN phone;

-- drop table/database
DROP TABLE enrollments;
DROP DATABASE university;
```

DML (Data Manipulation Language) statements (insert/update/delete/select)

- **INSERT** to add rows.
- **UPDATE** to change rows.
- **DELETE** to remove rows.

Examples:

```
-- insert (we used earlier)
INSERT INTO students (name,email) VALUES ('Alice', 'alice@example.com');

-- update
UPDATE students SET email = 'john.doe@example.com' WHERE student_id = 1;

-- delete
DELETE FROM enrollments WHERE enroll_id = 2;
```

Aggregate functions & GROUP BY / HAVING

Common aggregates: COUNT, SUM, AVG, MIN, MAX.

Examples:

```
-- number of students
SELECT COUNT(*) AS num_students FROM students;

-- average grade for all enrollments
SELECT AVG(grade) AS avg_grade FROM enrollments;

-- average grade per course
SELECT c.title, AVG(e.grade) AS avg_grade
FROM courses c
JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.title;

-- courses with average >= 85
SELECT c.title, AVG(e.grade) AS avg_grade
FROM courses c
JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id
HAVING AVG(e.grade) >= 85;
```

What is a JOIN?

A **JOIN** lets you combine rows from two or more tables **based on a related column** (usually a primary–foreign key relationship). This is the **heart of the relational model** — relationships across tables.

INNER JOIN

Returns **only the matching rows** between two tables.

Example: get students with the courses they enrolled in.

```
SELECT s.name, c.title, e.grade
FROM students s
INNER JOIN enrollments e ON s.student_id = e.student_id
INNER JOIN courses c ON e.course_id = c.course_id;
```

✅ Only students who are in **enrollments** will show up.

❌ Students without enrollments won't appear.

LEFT JOIN (a.k.a. LEFT OUTER JOIN)

👉 Returns **all rows from the left table**, plus matching rows from the right table. If no match, you get **NULL**.

Example: list all students and their courses (if any).

```
SELECT s.name, c.title, e.grade
FROM students s
LEFT JOIN enrollments e ON s.student_id = e.student_id
LEFT JOIN courses c ON e.course_id = c.course_id;
```

✅ Students without enrollments will still show, with **NULL** for course/grade.

RIGHT JOIN (RIGHT OUTER JOIN)

👉 Returns **all rows from the right table**, and matches from the left.

Example: list all courses, even if no student has enrolled.

```
SELECT s.name, c.title
FROM students s
RIGHT JOIN enrollments e ON s.student_id = e.student_id
RIGHT JOIN courses c ON e.course_id = c.course_id;
```

✅ Courses with zero students still show, but student name will be **NULL**.

FULL OUTER JOIN

👉 Returns **all rows from both sides**, with NULLs where there's no match.

⚠️ MySQL doesn't support **FULL JOIN** directly — you emulate it with **UNION**.

```
SELECT s.name, c.title
FROM students s
LEFT JOIN enrollments e ON s.student_id = e.student_id
LEFT JOIN courses c ON e.course_id = c.course_id
```

UNION

```
SELECT s.name, c.title
FROM students s
RIGHT JOIN enrollments e ON s.student_id = e.student_id
RIGHT JOIN courses c ON e.course_id = c.course_id;
```

LIVE SESSION EXPERIENCE SURVEY

**Before we proceed to Q&A
Take 2 minutes to share your feedback with Us!**

